Week 6: Benchmarking and Final Reporting

Hybrid QML and Quantum String Algorithms for Document Extraction

Pushkar Kumar • Mentor: Potluri Krishna Priyatham

Deliverable as specified in the project brief: Quantitative comparison against classical architectures; final report and presentation.

1 Objective

Compare compute resources against classical approaches, evaluate accuracy against model size, and produce the final report.

2 Experimental Design

Five experiments were defined, all reproducible from a committed dataset:

A. Feature-extractor comparison, per tier, at matched dimensionality.

B. End-to-end character error rate and identifier recovery per tier.

C. Quantum resource cost: encoding, quanvolutional layer, Grover oracle.

D. Grover oracle scaling with text length.

E. Shot-noise sensitivity of the quantum feature stage.

One design rule governs all of them: **when comparing quantum against classical, hold feature dimensionality fixed**. Otherwise a win is indistinguishable from granting one method a wider representation.

3 Experiment A: Feature Extractors

Table 1: Character accuracy by extractor and tier.

Tier	quanv (64)	quanv+ZZ (160)	classical (160)	raw pixels (64)
clean_digital	99.3%	100%	100%	100%
clean_scan	99.3%	100%	100%	100%
noisy_scan	93.0%	97.7%	97.7%	98.3%
clear_handwriting	87.7%	96.7%	96.3%	98.7%
degraded_handwriting	62.5%	75.7%	74.4%	79.4%
All tiers	**86.7%**	**92.2%**	**91.9%**	**93.6%**

3.1 Significance

A single split gives point estimates but cannot separate a real difference from noise. The comparison was repeated across 30 splits, with every extractor scored on the same splits so comparisons are paired.

Table 2: Paired comparisons across 30 splits.

Comparison	Difference	p	Verdict
quanv+ZZ − classical	−0.1 pt	≈ 0.2–0.3	not distinguishable from noise
raw pixels − quanv+ZZ	+1.2 pt	$< 10^{-10}$	significant
raw pixels − classical	+1.1 pt	$< 10^{-12}$	significant
quanv+ZZ − quanv	+5.7 pt	$< 10^{-25}$	significant

Figures are given as ranges because scikit-learn's solver is not bit-reproducible across BLAS builds: repeated runs on different machines give p between roughly 0.2 and 0.3 and win counts of 14 or 15 out of 30. The conclusions are stable across every run; the third decimal place is not, and quoting it would imply a precision this measurement does not have.

This separates two claims that the single-split table presents identically. The quantum-versus-classical gap is noise—the quantum layer wins roughly half the splits. The raw-pixel lead is not: it holds in all 30.

4 Experiment B: End-to-End

Table 3: End-to-end performance, all 12 documents per tier.

Tier	CER	Segmentation ratio	ID recovered exactly
clean_digital	6.6%	0.95	**100%**
clean_scan	8.5%	0.93	58%
noisy_scan	79.5%	0.34	0%
clear_handwriting	51.8%	0.70	0%
degraded_handwriting	85.7%	0.99	0%

The identifier column is the one that matters. Character error rate measures average quality; identifier recovery measures whether the pipeline did its job, for which one wrong character makes the result useless. The two diverge sharply on `clean_scan`: 8.5% CER but only 58% usable identifiers.

5 Experiments C and D: Resource Cost and Scaling

Table 4: Quantum resource cost.

Circuit	Qubits	Depth	CX gates
Quanvolutional filter (per patch)	4	10	8
Grover oracle, 16-character field	14	6,073	3,272

The filter runs 16 times per character, so the per-character cost is 128 CX gates; the per-document cost scales with character count.

Oracle scaling was measured across text lengths from 8 to 64 characters. Fitting a power law gives a CX exponent of **1.39**: superlinear, because each candidate position requires its own controlled data load. The measured curve sits above a linear reference and nowhere near $\sqrt{N}$.

Table 5: Accuracy against shot budget per patch.

Shots	Accuracy
64	70.7%
256	81.7%
1,024	85.2%
4,096	86.7%
exact	86.7%

6 Experiment E: Shot Noise

Approximately 1,024 shots per patch approach the noiseless result. At 16 patches per character this is roughly 16,000 circuit executions per character—a substantial hardware cost that belongs in any honest resource accounting and is frequently absent from published quanvolutional results.

7 External Baseline

Experiments A and B compare feature maps that all share this project's segmentation front end and classifier, which answers "which feature map is best inside this pipeline?" but leaves the pipeline unanchored. Tesseract provides the external reference, run on the same images over the same twelve documents per tier.

Table 6: Comparison against Tesseract.

Tier	This pipeline	Tesseract	Pipeline ID	Tesseract ID
clean_digital	6.6%	**2.0%**	**100%**	92%
clean_scan	8.5%	**1.9%**	58%	**83%**
noisy_scan	**79.5%**	100.0%	0%	0%
clear_handwriting	51.8%	**20.2%**	0%	**17%**
degraded_handwriting	**85.7%**	100.0%	0%	0%

Tesseract is three to four times better on clean input. That is the expected result and is reported plainly. Two details complicate rather than rescue the picture: on `clean_digital` this pipeline recovers more identifiers, because a constrained 36-class charset cannot emit out-of-charset characters; and on the two most degraded tiers Tesseract returns nothing at all, its page analysis rejecting input it judges unreadable.

This comparison is not controlled—Tesseract brings its own segmentation, language model and training corpus. It is included because the first question a sceptical reader asks is how the pipeline compares to an ordinary classical tool, and the answer belongs in the report regardless of whether it flatters the work.

8 Accuracy Against Model Size

The brief asked for accuracy against model size, in the spirit of comparing QNN parameters against LLM weights.

The quantum filter is the most parameter-efficient of the three that have parameters, producing a 160-dimensional feature space from 8 angles. But the zero-parameter option outperforms both, which is the finding: at this input size the parameter-efficiency comparison is won by not extracting features at all.

Table 7: Parameter count against accuracy.

Feature stage	Trainable parameters	Accuracy
Quanvolutional filter	8	91.7%
Classical convolution, dimension-matched	40	92.0%
Raw pixels	0	**93.6%**

9 Conclusions

1. At matched dimensionality the quantum feature layer is statistically indistinguishable from a classical convolution, and both lose to raw pixels. Training the filter does not change this.
2. Grover string matching works correctly, but oracle cost grows as $N^{1.39}$ owing to data loading, so the $\sqrt{N}$ query advantage does not become an end-to-end speedup for stored classical text.
3. The dominant error source is the classical segmentation front end, not either quantum stage.
4. Against Tesseract, this pipeline is three to four times worse on clean input. It is not competitive with mature classical OCR.
5. The single largest improvement came from correcting a train/serve data skew, not from any modelling change.

The work does not demonstrate quantum advantage for document intelligence, and identifies specific measured reasons why. Both are useful negative results, and both are more informative than a tuned demonstration would have been.

10 Reproducibility

Every figure regenerates from a clean repository clone. The dataset and trained model are committed; the browser demonstration is verified byte-identical to the Python implementation; and continuous integration re-runs the benchmark on every push and fails the build if any figure in the results file does not appear in the report. That final gate has caught a real contamination—a results file measured on a drifted dataset.

11 Deliverable Status

Complete. Five experiments implemented and measured, significance established, external baseline added, final report produced.

Artifacts: `benchmarking/run_benchmark.py`, `benchmarking/seed_sweep.py`, `benchmarking/tesseract_bas` `benchmarking/make_figures.py`, `docs/final_report.md`, `reports/qi26_25_final_report.pdf`